

**INSTITUT SUPÉRIEUR DES ARTS MULTIMÉDIA DE LA MANOUBA
(ISAMM)**

INTRODUCTION AU LANGAGE VHDL

Circuit logique programmable

Groupe : 2IM3

Mohamed Aziz Najlaoui

Mohamed Amine Yahia

Abdelkoddous Ben Younes

I. Introduction

VHDL (*VHSIC hardware description language*)

VHSIC (*Very High Integrated Circuits*)

Le **VHDL** est un langage de description matérielle. Contrairement aux langages logiciels classiques,

il est conçu pour décrire les circuits numériques, permettant leur conception et leur simulation avant l'implémentation physique dans des puces (FPGA ou ASIC).

VHDL est utilisé pour modéliser le matériel physique des systèmes numériques. Il permet un processus de conception plus fiable tout en minimisant les coûts et le temps de développement.

Important : VHDL n'est pas un langage de programmation comme JavaScript, Python ou C#. Lors de la conception d'un système VHDL, il est impératif de raisonner en termes de structure matérielle (logique câblée) et non en flux d'instructions logicielles.

II. Historique du VHDL

Année	Événement Marquant
1981	Initié par le département de la Défense (DoD) des États-Unis.
1983-85	Développement du langage par Intermetrics, IBM et TI.
1986	Transfert de tous les droits à l'IEEE.
1987	Publication du standard IEEE 1076-1987.
1994	Révision majeure du standard (VHDL 1076-1993).

Année	Événement Marquant
2002	Révision VHDL 1076-2002.
2009	Dernière révision majeure (VHDL 1076-2008).

III. Modèles de Description

- **Modèle Structurel** : Le système est décrit comme une interconnexion de portes et de blocs composants (similaire à un schéma électronique).
- **Modèle Comportemental** : Décrit le comportement d'un composant et l'interaction des signaux d'entrée pour produire la sortie.
- **RTL (Register Transfer Level)** : Décrit le flux de données entre les registres du système.
- **Modèle de Flux de Données (Dataflow)** : Définit le flux continu des données (ex: l'affectation $x \leq y$ est exécutée dès que y change).

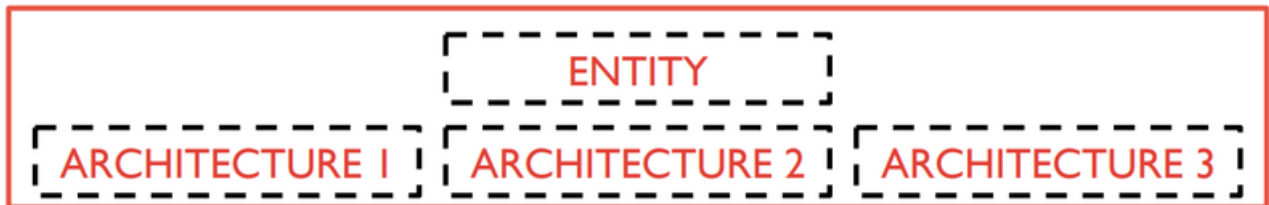
VI. Bibliothèque et Packages :

Avant de coder, il faut importer les bibliothèques standards contenant les types et opérateurs.

exemple :

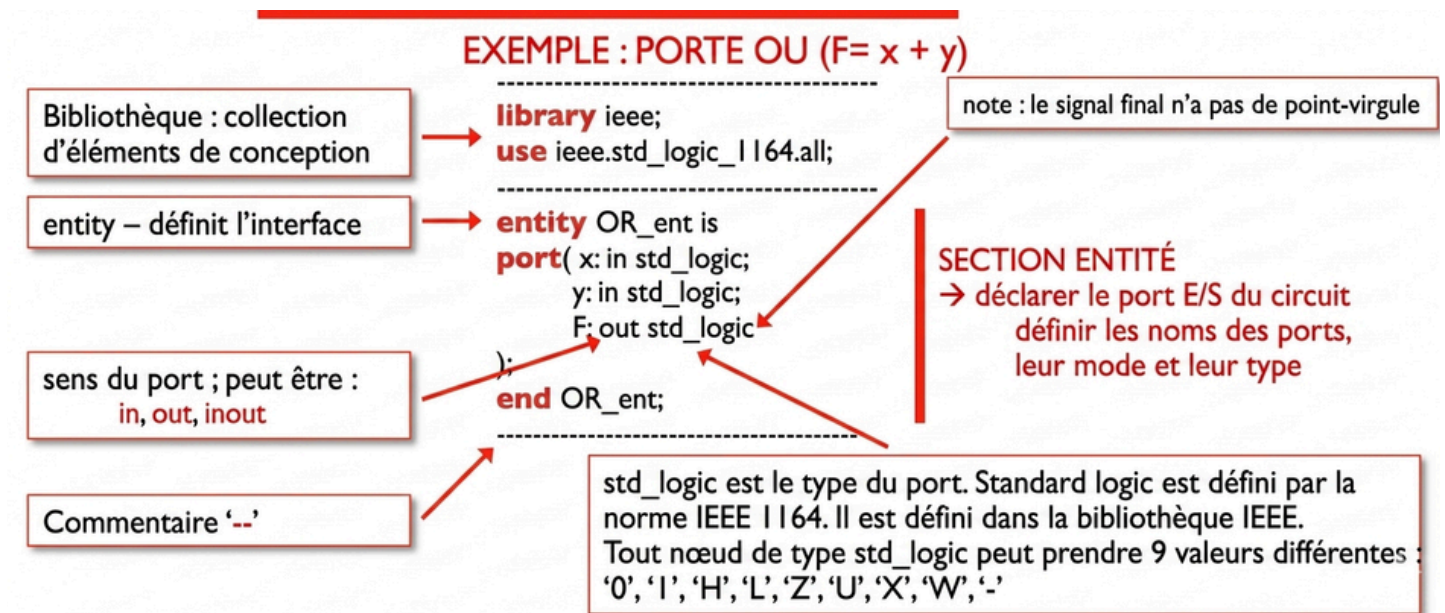
```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

V. Entité et Architecture



1. Les Entités (ENTITIES)

L'entité définit l'interface externe du composant matériel. Elle spécifie le nom de l'entité et les ports de communication (entrées et sorties).



2. Les Architectures (ARCHITECTURES)

L'architecture spécifie l'implémentation interne. Une seule entité peut posséder plusieurs architectures selon l'optimisation souhaitée (vitesse, surface ou puissance).

```
ARCHITECTURE nom_architecture OF  
  nom_entite IS BEGIN  
    -- Insertion du code VHDL ici  
  END nom_architecture;
```

3. Les Ports et Directions

Les noms de ports sont insensibles à la casse et doivent commencer par une lettre. Les directions possibles sont :

- **in** : Entrée (lecture seule).
- **out** : Sortie (écriture seule).
- **inout** : Bidirectionnel.
- **buffer** : Sortie spéciale pouvant être relue par l'architecture.

VI. Variables et Signaux

Bien que les deux contiennent des données, leur usage diffère radicalement :

- **Variables** : Utilisées uniquement à l'intérieur des processus (comportement séquentiel type logiciel). Elles sont mises à jour immédiatement avec l'opérateur `:=`.
- **Signaux** : Utilisés pour les descriptions structurelles. Ils sont mis à jour après un

délai de simulation avec l'opérateur `<=`

VII. Exemples de Code

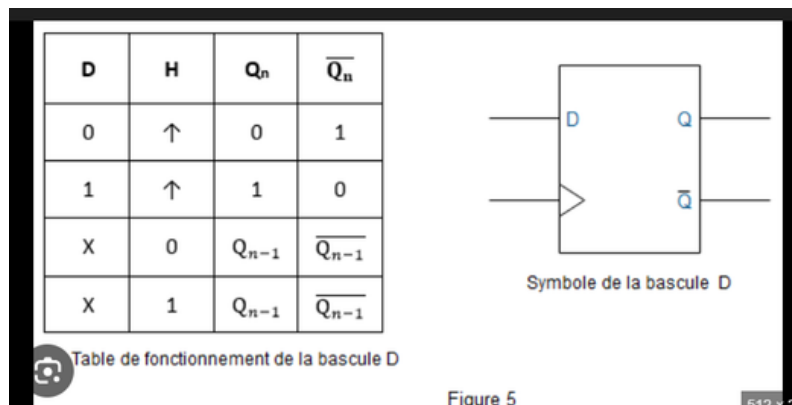
Porte ET (AND Gate)

```
library ieee;
use ieee.std_logic_1164.all;

entity AND_ent is
    port( x, y : in std_logic;
          F : out std_logic );
end AND_ent;

architecture behav1 of AND_ent is
begin
    process(x, y)
    begin
        if ((x='1') and (y='1')) then
            F <= '1';
        else
            F <= '0';
        end if;
    end process;
end behav1;
```

Bascule D (D Flip-Flop)



```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dff IS
    PORT( rst, clk, ena, d : IN std_logic;
          q : OUT std_logic );
END dff;

ARCHITECTURE synthesis1 OF dff IS
BEGIN
    PROCESS (rst, clk)
    BEGIN
        IF (rst = '1') THEN
            q <= '0';
        ELSIF (clk'EVENT) AND (clk = '1') THEN
            IF (ena = '1') THEN
                q <= d;
            END IF;
        END IF;
    END PROCESS;
END synthesis1;
```

Décompteur Incomplet Modulo 14

Ce décompteur commence sa séquence à partir de 13 (soit modulo 14) et descend vers 0.

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity decompteur_mod14 is
    port( clk, rst : in std_logic;
          count      : out std_logic_vector(3 downto 0) );
end decompteur_mod14;

architecture arch of decompteur_mod14 is
    signal temp: std_logic_vector(3 downto 0);
begin
    process(clk, rst)
    begin
        if rst = '1' then
            temp <= "1101"; -- Initialisation à 13 (D en hexa)
        elsif rising_edge(clk) then
            if temp = "0000" then
                temp <= "1101"; -- Reset à 13 si 0 atteint
            else
                temp <= temp - 1;
            end if;
        end if;
    end process;
    count <= temp;
end arch;

```

VIII. Conclusion

Le langage VHDL est un pilier de la conception de systèmes numériques modernes. Sa capacité à modéliser des comportements complexes tout en restant proche de la réalité matérielle en fait un outil indispensable pour les ingénieurs en informatique et électronique. La maîtrise des concepts d'entité, d'architecture et la distinction entre simulation et synthèse est essentielle pour réussir tout projet de design matériel robuste.